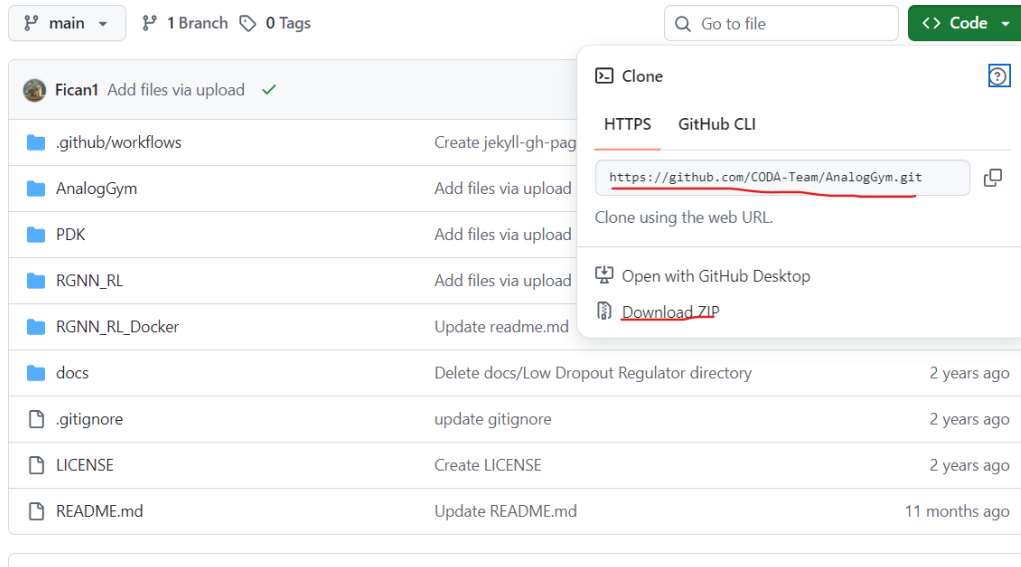


AnalogGym 详细配置与使用教程

一、克隆项目代码

<https://github.com/CODA-Team/AnalogGym>

1. 将 GitHub 上的代码克隆到本地机器上: git 到你的仓库或者本地下载 zip



二、通过 Conda 创建并激活虚拟环境

为了避免与其他 Python 项目的库版本冲突, 我们使用 Conda 创建一个干净的 Python 环境。具体的 anaconda 安装及 conda 创建环境教程自己网上查询

将 YAML 文件 (如 environment.yml) 放在当前工作目录中(RGNN_RL 目录)。在终端 (Anaconda Prompt) 中依次输入:

- # 1. conda env create -f environment.yml
- # 2. conda activate analoggym-env 激活虚拟环境

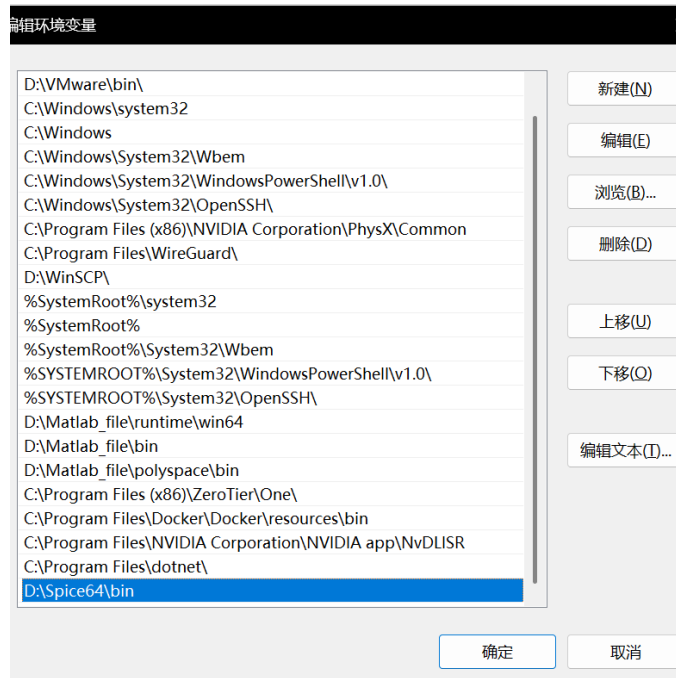
conda 常见命令集 <https://blog.csdn.net/miracleoa/article/details/106115730>

三、在 Windows 安装 Ngspice 仿真器

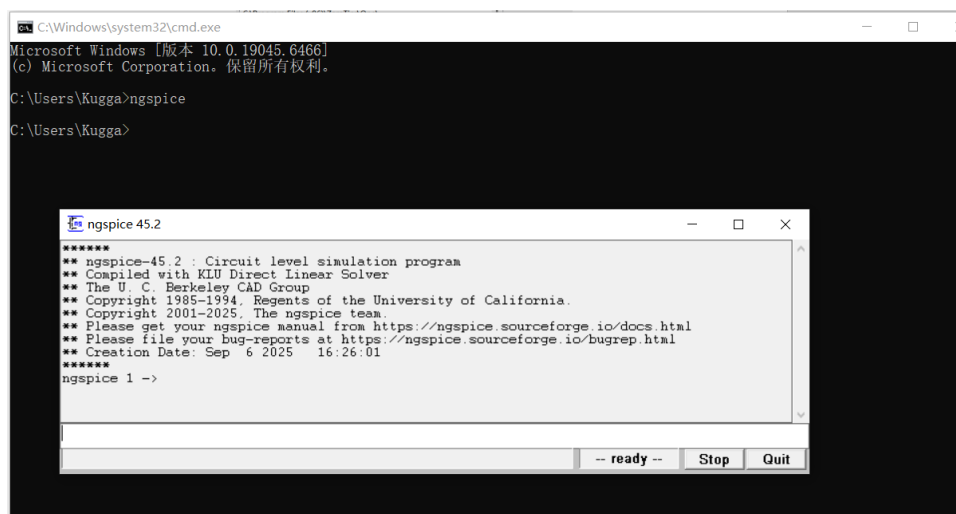
Ngspice 下载路径:

https://sourceforge.net/projects/ngspice/files/ng-spice-rework/45.2/ngspice-45.2_64.7z/download

1. 找个英文目录解压缩
2. 把 ngspice 文件夹里面的 bin 目录加载进 path:



2. cmd 里面能唤醒则代表安装成功:



四、代码报错修改

源代码可能存在部分版本不匹配的地方需要修改

1. 报错: rew_eng 不存在

```
18
19 from AMP_NMCF import AMPNMCFEnv
20
21 date = datetime.today().strftime('%Y-%m-%d')
22 PWD = os.getcwd()
23 SPICE_NETLIST_DIR = f'{PWD}/simulations'
24 os.environ['CUDA_LAUNCH_BLOCKING'] = "1"
25
26 CktGraph = GraphAMPNMCF
27 GNN = ActorCriticRGCN # you can select other GNN
28 rew_eng = CktGraph().rew_eng
29
```

问题出在：我们仿真的电路 AMP_NMCF 实际上是不含有 rew_eng，具体配置见 ckt_graphs.py 的 class GraphAMPNMCF。因为在代码实现中 rew_eng 现在不是核心逻辑变量，最快速的解决办法就是在 GraphAMPNMCF 中直接添加就行：

```
21 | | | # /
22 | | | self.rew_eng = True
```

不过部分其他电路是包含 rew_eng，因此视情况添加。

2. 找不到 sky130_pdk 工艺库：

RGNN_RL/mosfet_model 下的工艺库 sky130_pdk 没有解压，解压就行

3、电路无法正常获取 OP 仿真参数：

```
return env_reset_passive_checker(self.env, seed=seed, options=options)
File "D:\Anaconda\envs\gym01\lib\site-packages\gymnasium\utils\passive_env_checker.py", line 185, in env_reset_passive_checker
result = env.reset(**kwargs)
File "c:\Users\Kugga\桌面\AnalogGym-main\RGNN_RL\AMP_NMCF.py", line 123, in reset
observation = self._get_obs()
File "c:\Users\Kugga\桌面\AnalogGym-main\RGNN_RL\AMP_NMCF.py", line 211, in _get_obs
self.OP_M0 = self.op_results['M0']
TypeError: 'NoneType' object is not subscriptable
```

dev_params.py 下的 mos 晶体管 op 参数数量与 simulations/AMP_NMCF_dev_params.spice 规定的 mos 晶体管 op 参数数量不符。

这里建议直接替换为我改之后的 dev_params.py，或则可以参照 AMP_NMCF_dev_params.spice 注释 self.params_mos 的部分参数

```
110 | | | # 45 attributes for mos
111 | | | self.params_mos = (
112 | | |     'gmbs',
113 | | |     'gm',
114 | | |     'gds',
115 | | |     'vdsat',
116 | | |     'vth',
117 | | |     'id',
118 | | |     'ibd',
119 | | |     'ibs',
120 | | |     'gbd',
121 | | |     'gbs',
122 | | |     'isub',
123 | | |     'igidl',
124 | | |     'igisl',
125 | | |     'igs',
126 | | |     'igd',
127 | | |     'igb',
128 | | |     'igcs',
129 | | |     'vbs',
130 | | |     'vgs',
131 | | |     'vds',
132 | | |     'cgs',
133 | | |     'cgs',
134 | | |     'cgd',
135 | | |     'cgb',
136 | | |     'cbd',
137 | | |     'cbs',
138 | | |     'cdg',
139 | | |     'cdd',
140 | | |     'cgs',
141 | | |     'cgs',
142 | | |     'css',
143 | | |     'cgb',
144 | | |     'cbd',
145 | | |     'cbs',
146 | | |     'cbb',
147 | | |     'capbs',
148 | | |     'capbs',
149 | | |     # 'qg',
150 | | |     # 'qb',
151 | | |     # 'qs',
152 | | |     # 'qinv',
153 | | |     # 'qdef',
154 | | |     # 'gcrg',
155 | | |     # 'gtou'
156 | | | )
157 | | | )

let gmbs_M0=@m.x1.XM0.msky130_fd_pr_pfet_01v8[gmbs]
let gm_M0=@m.x1.XM0.msky130_fd_pr_pfet_01v8[gm]
let gds_M0=@m.x1.XM0.msky130_fd_pr_pfet_01v8[gds]
let vdsat_M0=@m.x1.XM0.msky130_fd_pr_pfet_01v8[vdsat]
let vth_M0=@m.x1.XM0.msky130_fd_pr_pfet_01v8[vth]
let id_M0=@m.x1.XM0.msky130_fd_pr_pfet_01v8[id]
let ibd_M0=@m.x1.XM0.msky130_fd_pr_pfet_01v8[ibd]
let ibs_M0=@m.x1.XM0.msky130_fd_pr_pfet_01v8[ibs]
let gbd_M0=@m.x1.XM0.msky130_fd_pr_pfet_01v8[gbd]
let gbs_M0=@m.x1.XM0.msky130_fd_pr_pfet_01v8[gbs]
let isub_M0=@m.x1.XM0.msky130_fd_pr_pfet_01v8[isub]
let igidl_M0=@m.x1.XM0.msky130_fd_pr_pfet_01v8[igidl]
let igisl_M0=@m.x1.XM0.msky130_fd_pr_pfet_01v8[igisl]
let igs_M0=@m.x1.XM0.msky130_fd_pr_pfet_01v8[igs]
let igd_M0=@m.x1.XM0.msky130_fd_pr_pfet_01v8[igd]
let igb_M0=@m.x1.XM0.msky130_fd_pr_pfet_01v8[igb]
let igcs_M0=@m.x1.XM0.msky130_fd_pr_pfet_01v8[igcs]
let vbs_M0=@m.x1.XM0.msky130_fd_pr_pfet_01v8[vbs]
let vgs_M0=@m.x1.XM0.msky130_fd_pr_pfet_01v8[vgs]
let vds_M0=@m.x1.XM0.msky130_fd_pr_pfet_01v8[vds]
let cgs_M0=@m.x1.XM0.msky130_fd_pr_pfet_01v8[cgs]
let cgd_M0=@m.x1.XM0.msky130_fd_pr_pfet_01v8[cgd]
let cbd_M0=@m.x1.XM0.msky130_fd_pr_pfet_01v8[cbd]
let cdg_M0=@m.x1.XM0.msky130_fd_pr_pfet_01v8[cdg]
let csg_M0=@m.x1.XM0.msky130_fd_pr_pfet_01v8[csg]
let cdb_M0=@m.x1.XM0.msky130_fd_pr_pfet_01v8[cbd]
```

五、项目运行

在 RGNN_RL 目录下，放大器的仿真直接运行 python main_AMP.py，同理 LDO 直接运行 python main_LDO.py

RL-Agent 的配置在两个 main 文件里都有

```
56
57 # parameters
58 num_steps = 10000
59 memory_size = 100000
60 batch_size = 128
61 noise_sigma = 2 # noise volume
62 noise_sigma_min = 0.1
63 noise_sigma_decay = 0.9995 # if 1 means no decay
64 initial_random_steps = 1000
65 noise_type = 'uniform'
66
67 agent = DDPGAgent(
68     env,
69     CktGraph(),
70     GNN().Actor(CktGraph()),
71     GNN().Critic(CktGraph()),
72     memory_size,
73     batch_size,
74     noise_sigma,
75     noise_sigma_min,
76     noise_sigma_decay,
77     initial_random_steps=initial_random_steps,
78     noise_type=noise_type,
79 )
```